

is also known as the control plane. Applications are deployed over worker nodes, specifically in pods. Pods are one or more containers that share volumes and a network name space, and are part of a single context.

The control plane is the commanding node of the Kubernetes cluster. It communicates with all the worker nodes of the cluster. It manages the various services of the worker nodes and makes global decisions about the cluster. The components of the control plane are listed below.

kube-apiserver: This exposes the Kubernetes API. The API server is the front end of the Kubernetes control plane.

etcd: This is a distributed, consistent key-value store used for configuration management, service discovery, and coordinating distributed work.

kube-scheduler: This watches for newly created pods with no assigned node, and binds them to a node to run on.

kube-controller-manager: This component manages all core components and makes the necessary changes in an attempt to move the current state towards the desired state.

cloud-controller-manager: This lets you link your cluster to your cloud provider's API.

The worker node is responsible for managing the running pods and enabling Kubernetes container runtime. Each node in the cluster runs a container runtime and the following components.

kubelet: This is responsible for running your application and reporting the status of the node to the API server in the master node.

kube-proxy: This runs on each node and is responsible for watching the API server for any changes while maintaining the entire network configuration up to date.

The Kubernetes cluster shown in Figure 1 outlines all the components

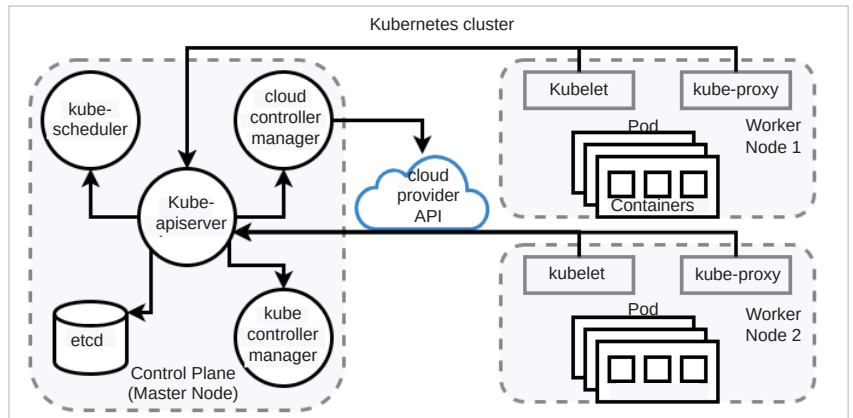


Figure 1: Kubernetes cluster architecture with manager node and worker nodes

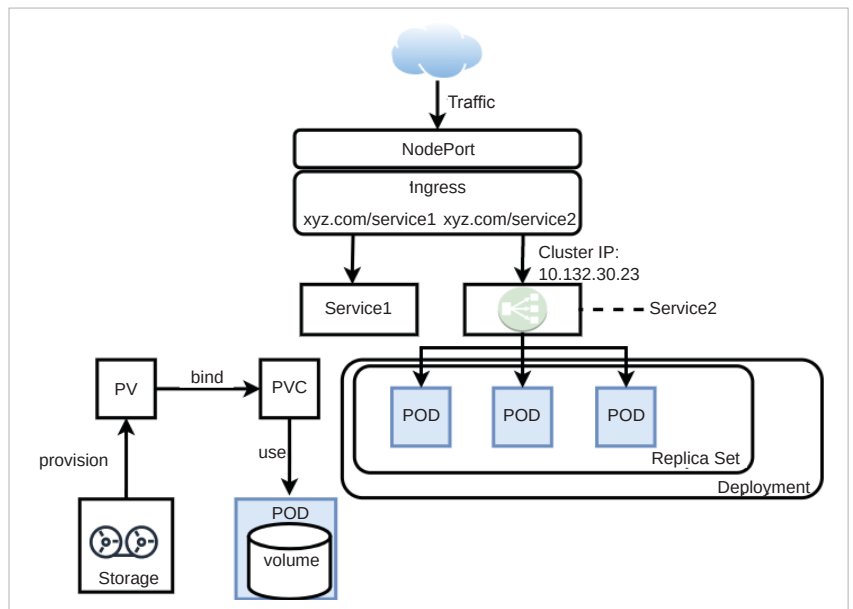


Figure 2: Synchronization and interaction of the various components of Kubernetes worker node

involved. The master node connects to the worker nodes to produce an environment to facilitate all the application pods. Kubelet in worker nodes reports the status via the exposed APIs of the master node while serving the pods within.

Some of the other important components representing the workload, network, and storage of a Kubernetes cluster are given below.

Pod: This is the smallest unit of work or management resource within Kubernetes. It is composed of one or more containers that share their storage, network, and context.

ReplicaSets: This is a method of managing pod replicas and their life cycle. It keeps the specified number of pods constant and is responsible for scheduling, scaling, and deletion of pods.

Deployment: This is a declarative method to manage various pods and their replica sets. With deployment, users can describe the life cycle of applications and images.

Cluster: The resources of this set of nodes are used to run a set of applications.

Jobs: These are best used to run a finite task to completion as opposed